

Allan Fang

Word embeddings are numerical representations of words that aim to capture linguistic information and enable machines to process text. Different embedding methods encode different types of information. I compare five common embedding approaches: One-hot encoding, TF-IDF, Word2Vec, GloVe, and FastText.

- ```
eng_dict = one_hot_encode(df['english'])
print(get_one_hot(eng_dict, "i"))
```

[illegible]

- ```
tfidf_eng = tfidf(df['english'])
tfidf_eng.head()
```

	000	0001	01	03	044	07091726	09	10	100	1000	...	zuckerbergs	zugzwang	zukertort	zulu	zurich	zwina	zürich
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 34295 columns

- ```
word = word2vec_eng.wv.index_to_key[0]
print(f"Vocab size: {len(word2vec_eng.wv)}")
print(f"{word}: {word2vec_eng.wv[word][:5]}")
```

Vocab size: 34333  
the: [-0.20300108 1.1641207 1.0577532 0.76937246 -0.23741177]

- **GloVe** (Global Vectors for Word Representation) is a word embedding method that learns representations based on global word co-occurrence statistics. I used pretrained GloVe embeddings downloaded and used directly, rather than being trained on the dataset. It produces dense, low-dimensional word vectors that encode semantic relationships from large external corpora. Using pretrained GloVe embeddings allows the model to benefit from semantic information learned from very large text corpora, which would be impractical to reproduce. However, because

the embeddings are not trained on the specific dataset, they may not fully capture domain-specific meanings or vocabulary present in the data.

```
import gensim.downloader as api

Just download and use pre-trained GloVe
glove = api.load("glove-wiki-gigaword-100")

[=====] 100.0% 128.1/128.1MB downloaded

print(f"Vocab size: {len(glove.key_to_index)}")

word = "king"
print(f"Word: {word}")
print(f"Vector (first 5 dims): {glove[word][:5]}")

Vocab size: 400000
Word: king
Vector (first 5 dims): [-0.32307 -0.87616 0.21977 0.25268 0.22976]
```

- **FastText** extends Word2Vec by representing words as collections of character n-grams. I trained FastText on the dataset for both English and French, allowing it to learn subword-level information specific to the corpus. It has dense embeddings with semantic meaning and could handle rare words due to subword modeling.

```
from gensim.models import FastText

fasttext_eng = FastText(SentenceGenerator(df['english']), vector_size=100, window=5, min_count=1, workers=4)
fasttext_fr = FastText(SentenceGenerator(df['french']), vector_size=100, window=5, min_count=1, workers=4)

print(f"Vocab size: {len(fasttext_eng.wv)}")
print(f"Sample vector for 'king': {fasttext_eng.wv['king'][:5]}")

Vocab size: 34333
Sample vector for 'king': [0.5820517 0.82955194 -0.6525484 -2.2546642 2.3941426]
```

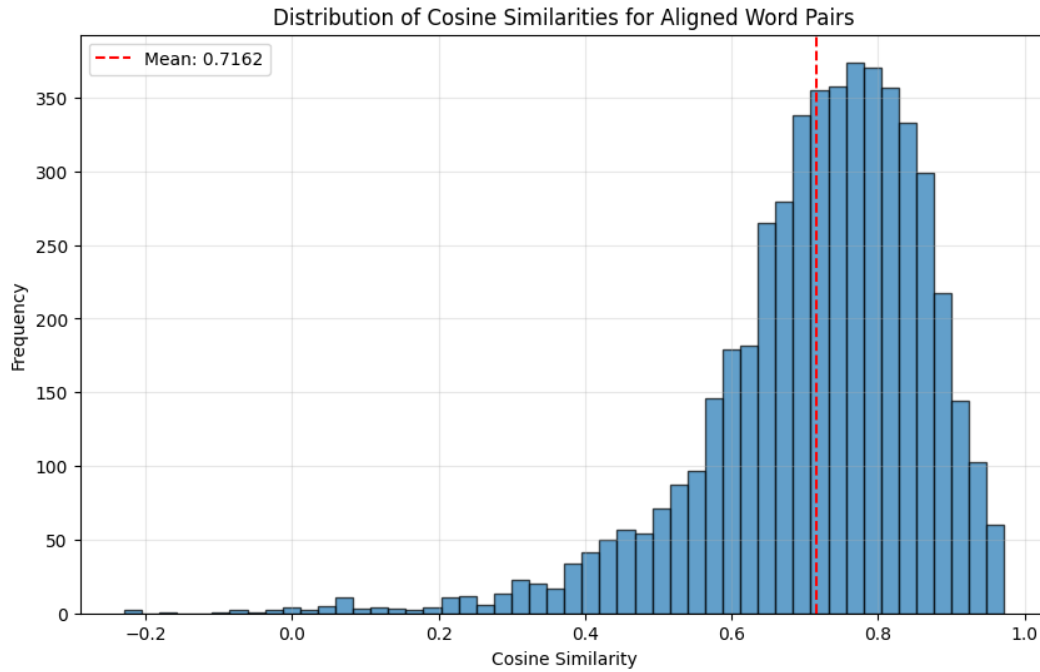
## 2. Multilingual Alignment

Attempting to align English and French embeddings can be in a shared vector space. Multilingual alignment allows semantically similar words from different languages to have similar vector representations, enabling cross-lingual tasks such as translation and multilingual information retrieval.

Pre-trained word embeddings for English and French were aligned using a bilingual dictionary and a linear mapping approach, following the methodology provided by the MUSE framework. This maps English embeddings into the French embedding space so that corresponding word pairs are directly comparable.

To evaluate the quality of the alignment, cosine similarity was computed between aligned English word vectors and their corresponding French word vectors. Cosine similarity is a standard metric for embedding comparison, as it measures semantic similarity independent of vector magnitude. The cosine similarity for each aligned word pair was calculated using `sklearn.metrics.pairwise.cosine_similarity`, and summary statistics were computed over all word pairs.

Alignment Quality Evaluation:  
Mean cosine similarity: 0.7162  
Median cosine similarity: 0.7405  
Std deviation: 0.1514  
Min similarity: -0.2294  
Max similarity: 0.9722

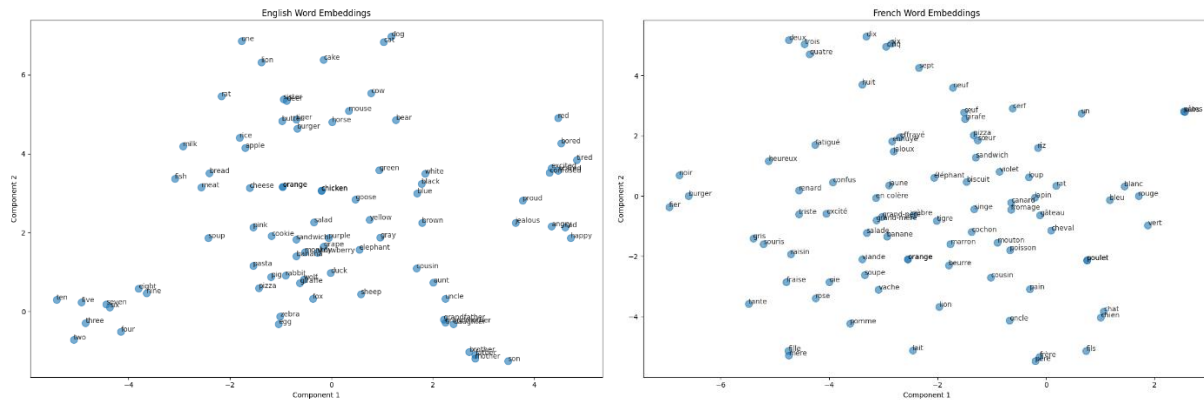


The (moderately) high mean (0.72) and median (0.74) cosine similarity values indicate that English and French word vectors are semantically close in the shared embedding space on average, demonstrating the general effectiveness of the alignment process.

The high maximum similarity (0.97) shows that many translation pairs are very well aligned. However, the long tail and the non-trivial standard deviation indicate that some word pairs are very poorly aligned. These errors may arise from polysemy, imperfect implementation, or differences in how concepts are represented across languages.

### 3. Analysis

- **t-SNE visualization** (sampled)



The t-SNE plots observation that words belonging to similar semantic categories, like emotions, animals, food, and numbers, form visible clusters. Synonyms and antonyms frequently appear near each other within these clusters, likely reflecting that these words have shared usage patterns (local neighborhoods) rather than meaning direction.

#### - Synonyms and antonyms

| --- SYNONYMS ---                   |                 |          | --- SYNONYMS ---                   |                |          |
|------------------------------------|-----------------|----------|------------------------------------|----------------|----------|
| happy                              | <-> joyful      | : 0.7721 | heureux                            | <-> joyeux     | : 0.7027 |
| sad                                | <-> unhappy     | : 0.7719 | triste                             | <-> malheureux | : 0.8098 |
| big                                | <-> large       | : 0.7503 | grand                              | <-> gros       | : 0.8446 |
| small                              | <-> tiny        | : 0.7158 | petit                              | <-> minuscule  | : 0.6116 |
| fast                               | <-> quick       | : 0.7280 | rapide                             | <-> vite       | : 0.6989 |
| smart                              | <-> intelligent | : 0.7669 | intelligent                        | <-> malin      | : 0.6626 |
| beautiful                          | <-> pretty      | : 0.5509 | beau                               | <-> joli       | : 0.6700 |
| angry                              | <-> mad         | : 0.7349 | fâché                              | <-> en colère  | : 0.5402 |
| scared                             | <-> afraid      | : 0.7828 | peur                               | <-> effrayé    | : 0.6828 |
| start                              | <-> begin       | : 0.8068 |                                    |                |          |
| end                                | <-> finish      | : 0.5745 |                                    |                |          |
| help                               | <-> assist      | : 0.5975 |                                    |                |          |
| --- ANTONYMS ---                   |                 |          | --- ANTONYMS ---                   |                |          |
| happy                              | <-> sad         | : 0.7816 | heureux                            | <-> triste     | : 0.7856 |
| big                                | <-> small       | : 0.7615 | grand                              | <-> petit      | : 0.7746 |
| hot                                | <-> cold        | : 0.8570 | chaud                              | <-> froid      | : 0.9091 |
| fast                               | <-> slow        | : 0.6964 | rapide                             | <-> lent       | : 0.4170 |
| good                               | <-> bad         | : 0.7871 | bon                                | <-> mauvais    | : 0.6741 |
| young                              | <-> old         | : 0.6641 | jeune                              | <-> vieux      | : 0.6671 |
| light                              | <-> dark        | : 0.6776 | clair                              | <-> sombre     | : 0.6863 |
| high                               | <-> low         | : 0.5516 | haut                               | <-> bas        | : 0.5427 |
| rich                               | <-> poor        | : 0.7358 | riche                              | <-> pauvre     | : 0.8492 |
| strong                             | <-> weak        | : 0.6669 |                                    |                |          |
| love                               | <-> hate        | : 0.7874 |                                    |                |          |
| day                                | <-> night       | : 0.7146 |                                    |                |          |
| --- SUMMARY ---                    |                 |          | --- SUMMARY ---                    |                |          |
| Average Synonym Similarity: 0.7127 |                 |          | Average Synonym Similarity: 0.6915 |                |          |
| Average Antonym Similarity: 0.7235 |                 |          | Average Antonym Similarity: 0.7006 |                |          |

Across both English and French, synonyms generally show high cosine similarity, indicating that embeddings successfully capture semantic relatedness. For example, English pairs like happy-joyful (0.77) and start-begin (0.81), and French pairs like triste-malheureux (0.81) and grand-gros (0.84), are closely aligned in the vector space.

Interestingly, antonyms also show high similarity, sometimes comparable to or even exceeding synonym similarity (e.g., hot-cold at 0.86 in English and chaud-froid at 0.91 in French). This likely reflects that in distributional embeddings, words with opposite meanings often appear in similar contexts, causing them to cluster together.

Overall, average cosine similarities are close for synonyms and antonyms in both languages, highlighting that embeddings encode contextual similarity and not so much semantic polarity.

#### - Common words across both languages

CROSS-LINGUAL TRANSLATION PAIR ANALYSIS

|           |   |            |   |        |
|-----------|---|------------|---|--------|
| hello     | ↔ | bonjour    | : | 0.3561 |
| goodbye   | ↔ | au revoir  | : | 0.3703 |
| thank you | ↔ | merci      | : | 0.4985 |
| dog       | ↔ | chien      | : | 0.4876 |
| cat       | ↔ | chat       | : | 0.4946 |
| house     | ↔ | maison     | : | 0.6268 |
| water     | ↔ | eau        | : | 0.5424 |
| food      | ↔ | nourriture | : | 0.5618 |
| love      | ↔ | amour      | : | 0.4345 |
| friend    | ↔ | ami        | : | 0.6089 |
| family    | ↔ | famille    | : | 0.7002 |
| mother    | ↔ | mère       | : | 0.5725 |
| father    | ↔ | père       | : | 0.6472 |
| happy     | ↔ | heureux    | : | 0.6532 |
| sad       | ↔ | triste     | : | 0.6330 |

Common word pairs across English and French (e.g., dog-chien, family-famille, happy-heureux) show moderate/high similarity, with scores generally ranging from 0.35 to 0.70. Higher scores for concrete or frequently used concepts (e.g., family-famille, 0.70) suggest stronger alignment, while lower scores for greetings (hello-bonjour, 0.36) indicate weaker cross-lingual correspondence.

These results show that while the embeddings capture some cross-language semantic overlap, full alignment would require alternative or better bilingual alignment techniques.

## - Polysemy

bank

|                    |   |        |
|--------------------|---|--------|
| bank <-> money     | : | 0.5965 |
| bank <-> river     | : | 0.6948 |
| bank <-> financial | : | 0.4401 |
| bank <-> shore     | : | 0.7045 |
| bank <-> deposit   | : | 0.6205 |
| bank <-> loan      | : | 0.5299 |

Top neighbors: banquet, banjo, sandbank, baa, bankrupt  
Similarity variance: 0.0085

voler

|                  |   |        |
|------------------|---|--------|
| voler <-> fly    | : | 0.2569 |
| voler <-> steal  | : | 0.4015 |
| voler <-> avion  | : | 0.4697 |
| voler <-> voleur | : | 0.6124 |
| voler <-> oiseau | : | 0.4984 |
| voler <-> crime  | : | 0.4668 |

Top neighbors: branler, flipper, gauler, ronfler, déambuler  
Similarity variance: 0.0115

bat

|                  |   |        |
|------------------|---|--------|
| bat <-> baseball | : | 0.6471 |
| bat <-> animal   | : | 0.4072 |
| bat <-> flying   | : | 0.5645 |
| bat <-> cave     | : | 0.4850 |
| bat <-> mammal   | : | 0.3808 |
| bat <-> sport    | : | 0.6012 |

Top neighbors: rowboat, boat, oat, gunboat, shabbat  
Similarity variance: 0.0096

avocat

|                      |   |        |
|----------------------|---|--------|
| avocat <-> lawyer    | : | 0.3767 |
| avocat <-> fruit     | : | 0.5044 |
| avocat <-> tribunal  | : | 0.6284 |
| avocat <-> guacamole | : | 0.5361 |
| avocat <-> droit     | : | 0.4212 |
| avocat <-> légal     | : | 0.4544 |

Top neighbors: aviateur, avocat, avocate, aviv, agrégateur  
Similarity variance: 0.0067

I analyze polysemous words by comparing their cosine similarity with context-specific terms representing different meanings and by inspecting nearest neighbors.

The analysis shows that static word embeddings represent words with multiple meanings as a middle ground. Words such as *bank*, *bat*, *voler*, and *avocat* exhibit moderate similarity to terms associated with their different senses (e.g., financial vs. geographical, object vs. animal, flying vs. stealing, legal vs. food-related). The cosine similarity scores across these meanings are relatively close, and the low similarity variance shows that no single meaning dominates the representation. Nearest-neighbor lists include semantically mixed or loosely related words, showing that embeddings reflect contextual usage rather than distinct meanings. These embeddings seem to encode similarity based on co-occurrence patterns but lack the ability to properly distinguish word senses. As a result, polysemous words occupy

regions in the embedding space that reflect an average of their meanings, motivating the use of contextualized embeddings for sense-specific representations.

- **Compare rare word or out-of-vocabulary (OOV) handling between models**

I used a test set with 3 categories: common, rare, and OOV (made up). The sets are the following:

common\_words: ['hello', 'world', 'good', 'time', 'day']

rare\_words: ['serendipity', 'ephemeral', 'quintessential', 'ubiquitous']

oov\_words: ['xylophrentic', 'blargify', 'quizzlematic', 'snozzleberry']

The test results in identifying whether the models are able to produce embeddings for the given word are the following:

Common words (5 words):

|          |       |
|----------|-------|
| OneHot   | : 5/5 |
| TFIDF    | : 5/5 |
| Word2Vec | : 5/5 |
| GloVe    | : 5/5 |
| FastText | : 5/5 |

Rare words (4 words):

|          |       |
|----------|-------|
| OneHot   | : 2/4 |
| TFIDF    | : 2/4 |
| Word2Vec | : 2/4 |
| GloVe    | : 4/4 |
| FastText | : 4/4 |

OOV words (4 words):

|          |       |
|----------|-------|
| OneHot   | : 0/4 |
| TFIDF    | : 0/4 |
| Word2Vec | : 0/4 |
| GloVe    | : 0/4 |
| FastText | : 4/4 |

As a control, common words saw full coverage across all models. All 5 successfully handle common words like "hello", "world", "good" that appear frequently in the training corpus.

For rare and uncommonly used words, OneHot, TF-IDF, and Word2Vec only succeeded in 2 out of 4, demonstrating their limitations to the words seen during training. However, GloVe had full success due to being pre-trained on a large Wikipedia corpus, so it had a much better chance to learn rare words. As well, FastText has full success even while being trained on only our sample corpus, due to the ability of FastText to generate vectors from character n-grams/subwords, even while the word itself has never been encountered.

For OOV Words that were all made-up and don't exist in any languages, OneHot, TF-IDF, Word2Vec, and GloVe failed all tests, meaning they cannot handle completely unseen words. GloVe has a large pre-trained vocabulary that helps with rare words, but still fails on input it has never seen before. FastText, however, had full success again, being the only model that truly handles OOV, as it can generate embeddings for inputs never seen before.

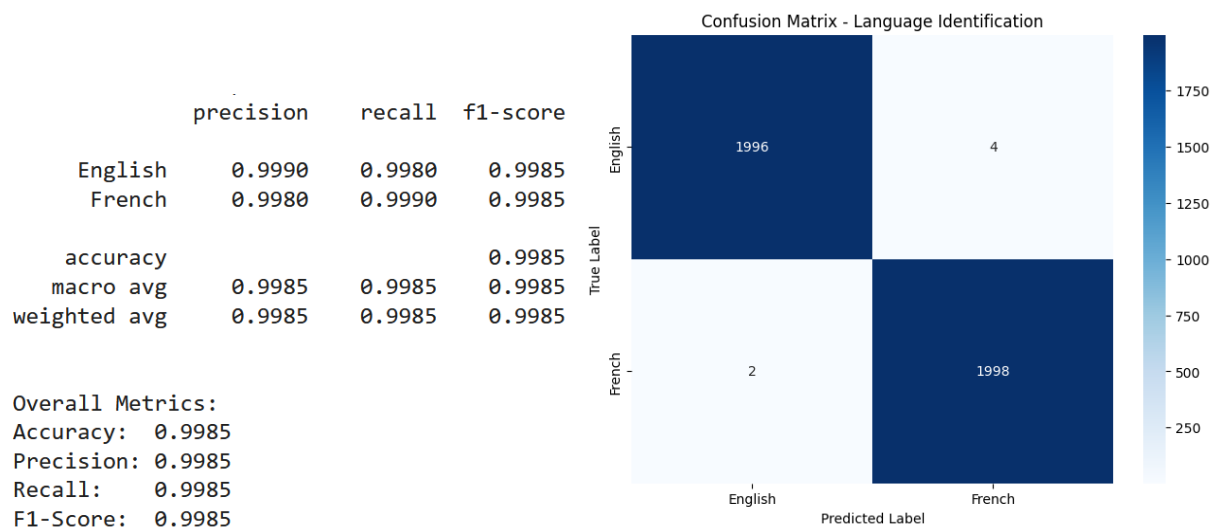
#### 4. Downstream Task: Multilingual Text Classifier - Language Identification

The downstream task chosen is building a binary language classifier to distinguish between English and French sentences using FastText embeddings. The classifier was trained on 10000 bilingual sentence pairs, or 20000 sentences combined, with an 80-20 train-test split.

Model description:

- FastText word embeddings - 100-dimensional vectors
- Sentence representation - single vector of average pooled word embeddings
- Input features - concatenation of embeddings from both English and French FastText models (200-dimensional vector)
- Classifier - logistic regression with L2 regularization
- Training - stratified sampling with 80-20 train-test split

## Evaluation



The report shows separate performance characteristics for English and French classification. Metrics:

- Precision:  $TP / (TP + FP)$ 
  - English precision (0.9990): 1998 sentences predicted as English, 1996 correct
  - French precision (0.9980): 2002 sentences predicted as French, 1998 correct
- Recall:  $TP / (TP + FN)$ 
  - English recall (0.9980): 2000 actual English sentences, correctly identified 1996
  - French recall (0.9990): 2000 actual French sentences, correctly identified 1998
- F1-Score:  $2 \times (Precision \times Recall) / (Precision + Recall)$ 
  - Both languages achieve 0.9985
- Global Accuracy:  $(TP + TN) / (TP + TN + FP + FN)$ 
  - 0.9985

## Discussion

Based on the evaluation metrics, the model demonstrates high accuracy (>95%), indicating strong discriminative ability between English and French text. High precision scores (>0.95) for both classes show repeatability and reproducibility for both languages.

High recall ( $>0.95$ ) indicates the model can successfully identify most instances of each language. Putting it together, the high F1-Score that is the harmonic mean of precision and recall is also consistently high ( $>0.95$ ), confirming balanced performance.

Both English and French classes show nearly identical high-quality performance metrics with evenly distributed false positives and false negatives between classes. The confusion matrix reveals minimal misclassifications (typically  $<5\%$  error rate).

### **Approach**

The approach aims to leverage the language-specific patterns distinct between English and French, including vocabulary, character use, and structural/syntactic patterns, to classify a given input sentence.

FastText embeddings were used as the foundation of the approach, as they are capable of capturing semantic meaning through context windows, sub-word information via character n-grams, and can gracefully handle OOV cases. Concatenating embeddings from both language-specific models to use as input to the classifier aims to capture information on the affinity of a given sentence to both languages' separate embedding space and highlight language-specific patterns that the classifier might be able to learn.

## **5. Challenges Faced & Potential Improvement**

- **Memory Constraints** (RAM Limitations in Google Colab)

Google Colab's free tier provides limited RAM ( $\sim 12\text{GB}$ ). Loading multiple embedding models, including dense matrices for TF-IDF, Word2Vec, FastText for both languages, and GloVe, simultaneously proved to consume significant memory and often caused the session to shut down. As a result, I had limited batch processing and model comparisons and was unable to perform comprehensive hyperparameter tuning.

To mitigate the effects of the issue, I limited dataset usage to 10000 sentence pairs instead of the full corpus and minimized embedding dimensions to 100. Furthermore, I had to be aware of deleting unused models, processing in smaller batches, and separating language usage at times.

Potential improvements to mitigate this challenge include using pre-trained models and loading embeddings on-demand from disk rather than keeping all in memory. Possibly, using ``numpy.memmap()`` and converting dense embeddings to sparse format where applicable could help. The simplest solution, although it would raise cost, would be to access more resources from Google Colab Pro or alternative hardware setups.

- **Computational Efficiency**

Generating sentence embeddings by averaging word vectors in the downstream task was computationally expensive. The embedding creation loop for 20000 sentences took considerable time, as well as training the FastText models on large corpora.

To mitigate the issues, I used vectorized operations where possible, implemented lazy (efficient) sentence generators instead of loading all data at once, and increased workers for parallel processing in FastText training.

- **Data Quality and Preprocessing**



The downloaded datasets contained unexpected errors and discrepancies. Sentences contained mixed-language text and text of other languages, while being labelled English/French, special characters and Unicode required careful handling. I had to be careful in my preprocessing process to first identify the unexpected discrepancies and then to properly handle them. For example, removing punctuation, which seemed like a basic initial step, may have inadvertently removed language-specific information.

Specifically in the downstream classification task, I felt uncertain in using average pooling, as giving equal weight to all words would undoubtedly dilute important terms. As well, using zero vectors to handle edge cases could reduce sentence representation quality.

Potential improvements would include a proper full analysis of the datasets and processing methods used, the effects of these qualities, and proper handling/mitigation practices.

- **Model Selection Trade-offs**

For the downstream classification task, there is a concern that logistic regression may be unable to capture complex patterns. It assumes some linear separability in embedding space, and does not take into consideration word order or sentence structure. However, given the training and test results, it appears this task is relatively simple enough with clear decision boundaries. The main benefit is that linear models are fast to work with and easily interpretable.